Decision Tree

Instructor: Jyotishka Datta [1]

Virginia Tech

March 25, 2025

[1] Email: jyotishka@vt.edu

Decision Tree

Decision Trees

- Tree-based methods for regression and classification.
- These involve **stratifying or segmenting** the predictor space into a number of simple regions.
- In order to make a prediction for a given observation, we typically use the mean or the mode of the training observations in the region to which it belongs.

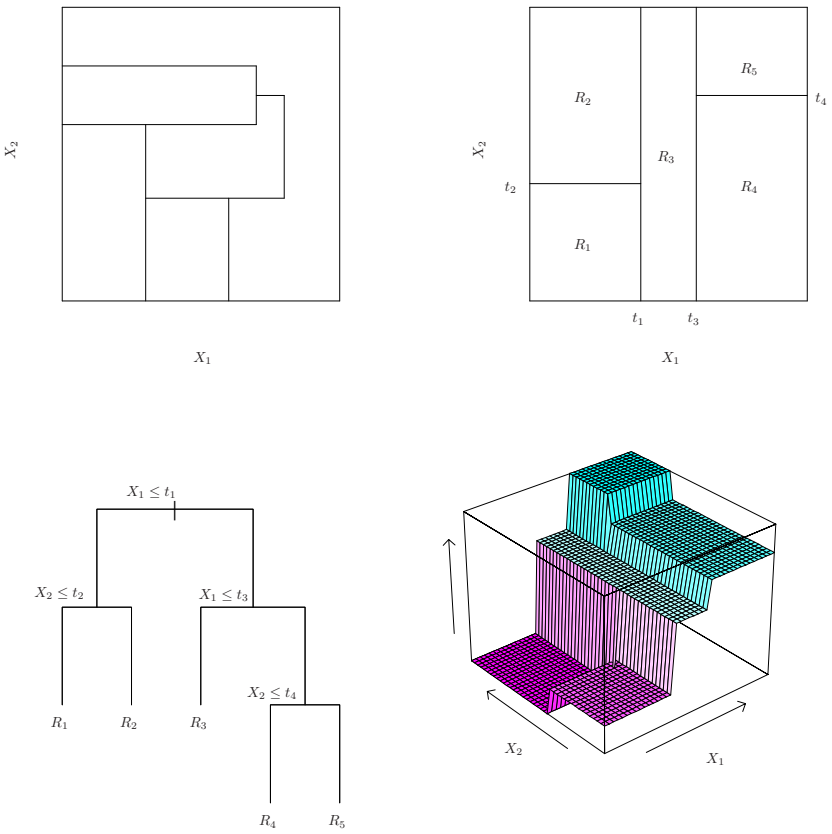
Decision Trees

- Tree-based methods for regression and classification.
- These involve **stratifying or segmenting** the predictor space into a number of simple regions.
- In order to make a prediction for a given observation, we typically use the mean or the mode of the training observations in the region to which it belongs.
- The set of splitting rules used to segment the predictor space can be summarized in a tree – decision tree.

Decision Trees

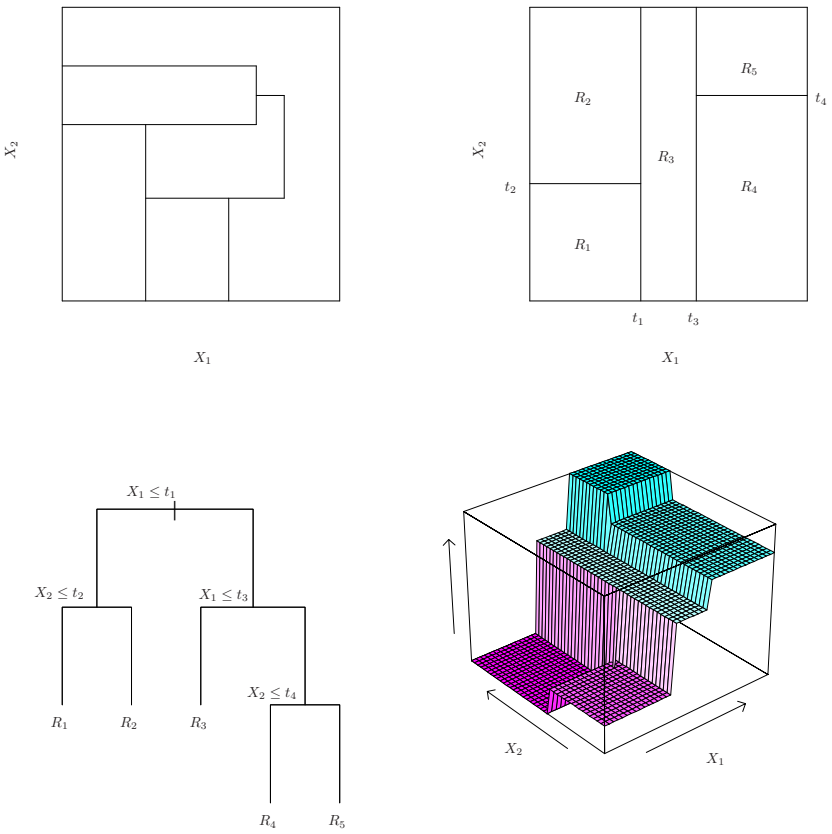
- Tree-based methods are simple and useful for interpretation.
- However, they typically are not competitive with the best supervised learning approaches like Lasso in terms of prediction accuracy.
- Leads to bagging, random forests, and boosting.
- Each of these approaches involves producing multiple trees which are then combined to yield a single **consensus prediction**.
- Combining a large number of trees can often result in dramatic improvements in prediction accuracy, at the expense of some loss in interpretation.

10,000 feet view



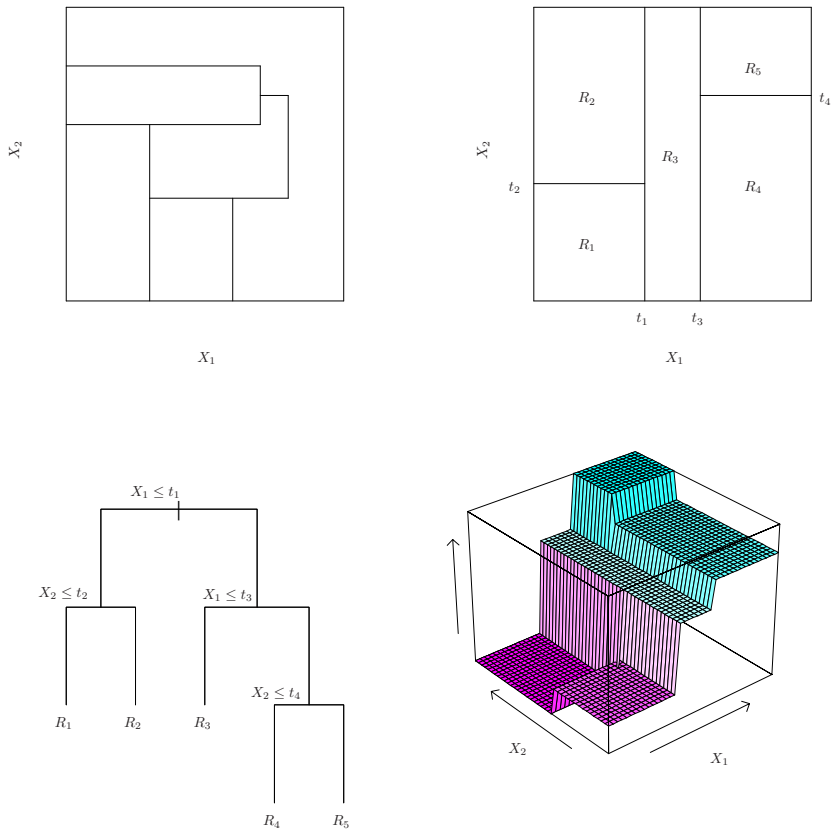
- Find a partition of the space of predictors.

10,000 feet view



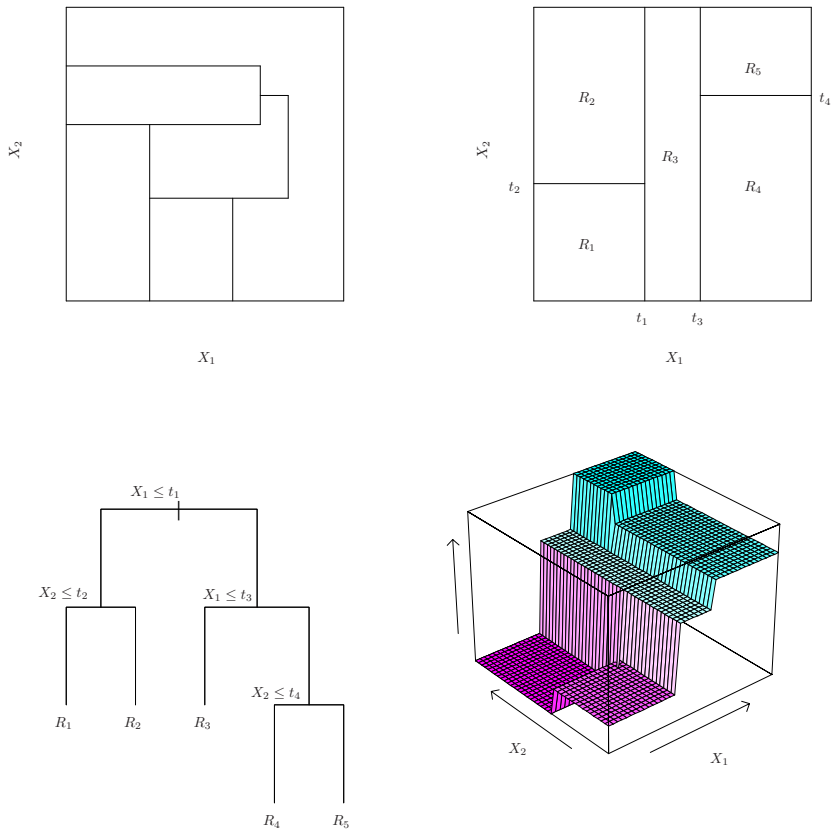
- Find a partition of the space of predictors.
- Predict a constant in each set of the partition.

10,000 feet view



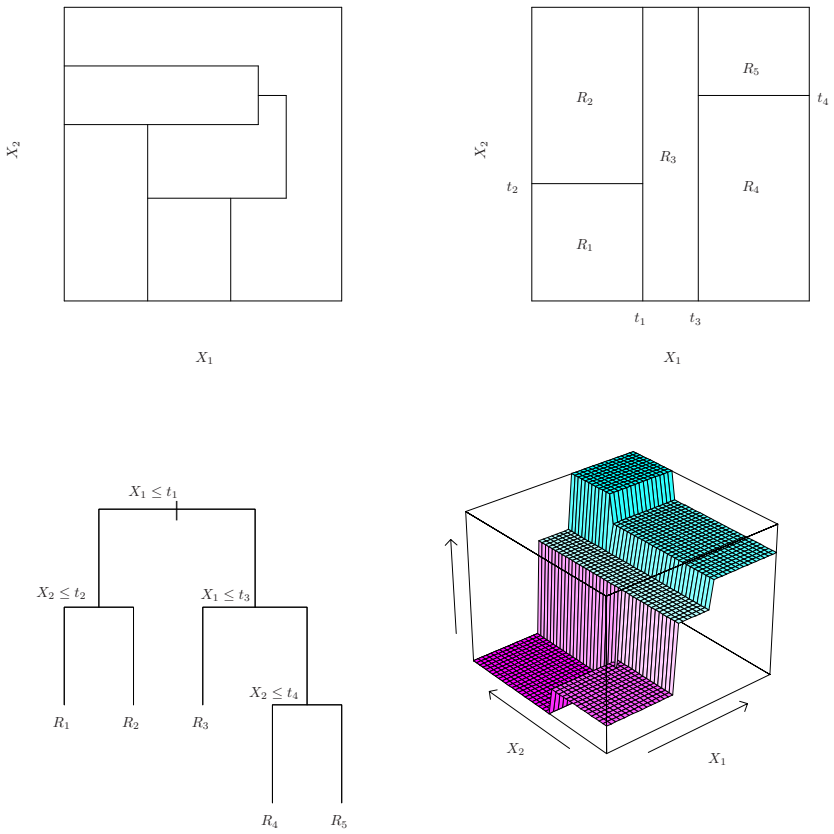
- Find a partition of the space of predictors.
- Predict a constant in each set of the partition.
- The partition is defined by splitting the range of one predictor at a time.

10,000 feet view



- Find a partition of the space of predictors.
- Predict a constant in each set of the partition.
- The partition is defined by splitting the range of one predictor at a time.
- Can be represented as a decision tree.

10,000 feet view



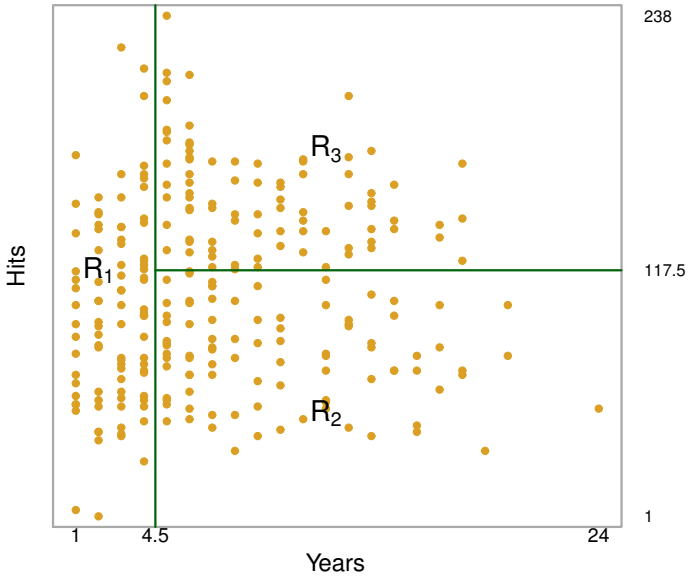
- Find a partition of the space of predictors.
- Predict a constant in each set of the partition.
- The partition is defined by splitting the range of one predictor at a time.
- Can be represented as a decision tree.
- Not all partitions are possible.

Example: Predicting a baseball player's salary

We use the `Hitters` data set to predict a baseball player's `Salary` based on `Years` (the number of years that he has played in the major leagues) and `Hits` (the number of hits that he made in the previous year).
We first remove observations that are missing `Salary` values, and log-transform `Salary` so that its distribution has more of a typical bell-shape.
(Recall that `Salary` is measured in thousands of dollars.)

Example: Predicting a baseball player's salary

The prediction for a point in region R_i is the average of the training points in R_i.



How is a decision tree built?

- Start with a single region R_1 (entire input space), and iterate:

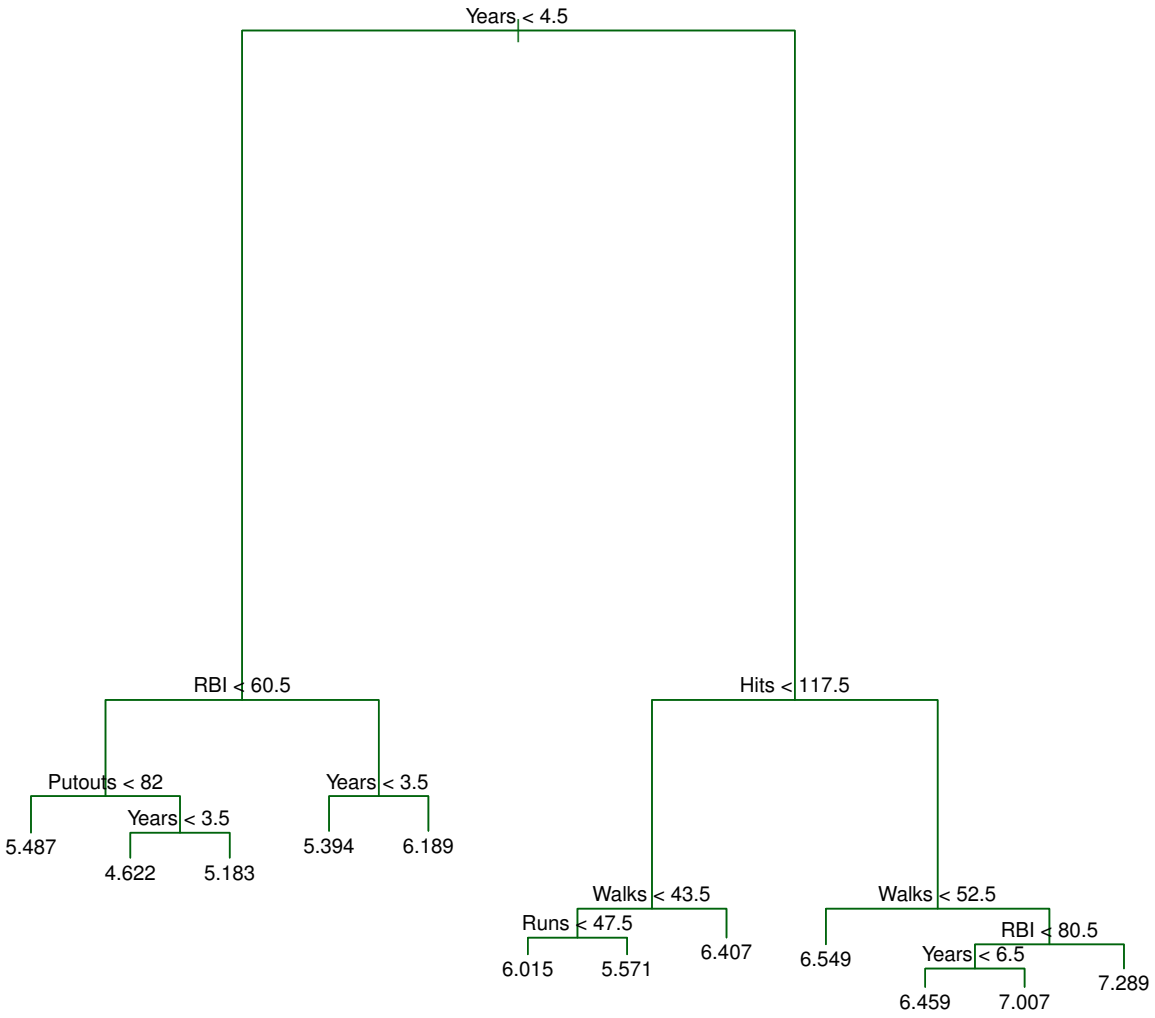
  1. Select a region R_k, a predictor X_j, and a splitting point s, such that splitting R_k with the criterion $X_j < s$ produces the largest decrease in RSS:

$$\sum_{m=1}^{|T|} \sum_{x_i \in R_m} (y_i - \bar{y}_{R_m})^2$$

  2. Redefine the regions with this additional split.

- Terminate when there are 5 observations or fewer in each region (or use a different stopping criterion.)
- This grows the tree from the root towards the leaves (Top-down greedy approach).

A decision tree for baseball salaries



How do we control overfitting?

- **Idea 1:** Find the optimal subtree by cross validation.

  $\rightarrow$ There are too many possibilities, so we would still over fit.

- **Idea 2:** Stop growing the tree when the RSS doesn't drop by more than a threshold with any new cut.

  $\rightarrow$ In our greedy algorithm, it is possible to find good cuts after bad ones.

How do we control overfitting?

Solution: Prune a large tree from the leaves to the root.

- **Weakest link pruning:**
  - Starting with with the initial full tree T_0, replace a subtree with a leaf node to obtain a new tree T_1. Select subtree to prune by minimizing:

$$\frac{RSS(T_1) - RSS(T_0)}{|T_0| - |T_1|}.$$

  - Iterate this pruning to obtain a sequence $T_0, T_1, T_2, \ldots, T_m$ where T_m is the tree with a single leaf node.
  - Select the optimal tree T_i by cross validation.

How do we control overfitting?

... or an equivalent procedure

- **Cost complexity pruning:**
  - Minimize the following objective over all prunings T of T_0:

$$\text{minimize} \sum_{R_m \in T} \sum_{x_i \in R_m} (y_i - \bar{y}_{R_m})^2 + \alpha |T|.$$

  - When $\alpha = \infty$, we select the null tree (=tree with one leaf node.)
  - When $\alpha = 0$, we select the full tree.
  - Fun fact: The solution for each α is among $T_1, T_2, \ldots, T_m$ from weakest link pruning.
  - Choose the optimal α (the optimal T_i) by cross validation.

Cross validation, the wrong way

1. Construct a sequence of trees $T_0, \ldots, T_m$ for a range of values of α.
2. Split the training points into 10 folds.
3. For $k = 1, \ldots, 10$,
   - For each tree T_i, use every fold except the kth to estimate the averages in each region.
   - For each tree T_i, calculate the RSS in the test fold.
4. For each tree T_i, average the 10 test errors, and select the value of α that minimizes the error.

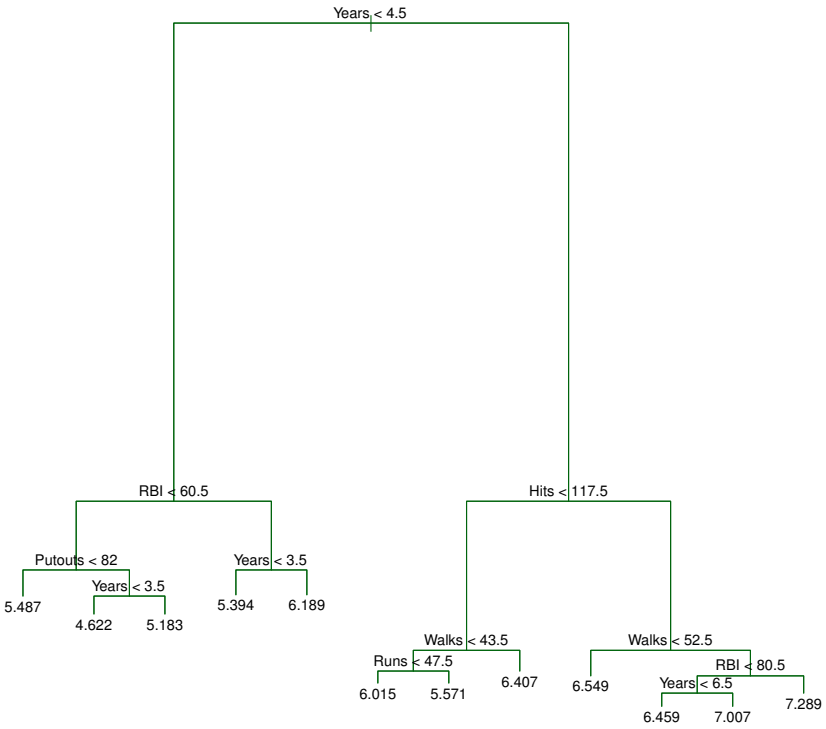
WRONG WAY TO DO CROSS VALIDATION!

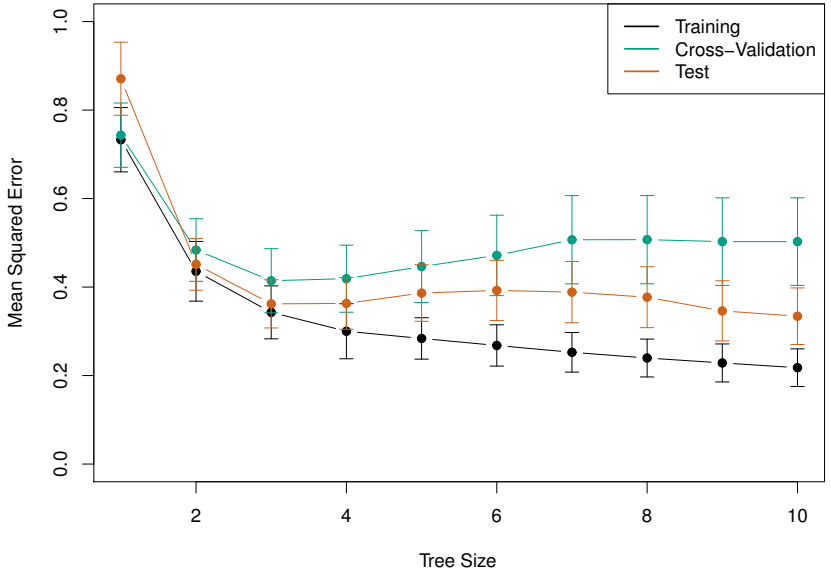
Cross validation, the right way

1. Split the training points into 10 folds.
2. For $k = 1, \ldots, 10$, using every fold except the kth:
   - Construct a sequence of trees $T_1, \ldots, T_m$ for a range of values of α, and find the prediction for each region in each one.
   - For each tree T_i, calculate the RSS on the test set.
3. Select the parameter α that minimizes the average test error.

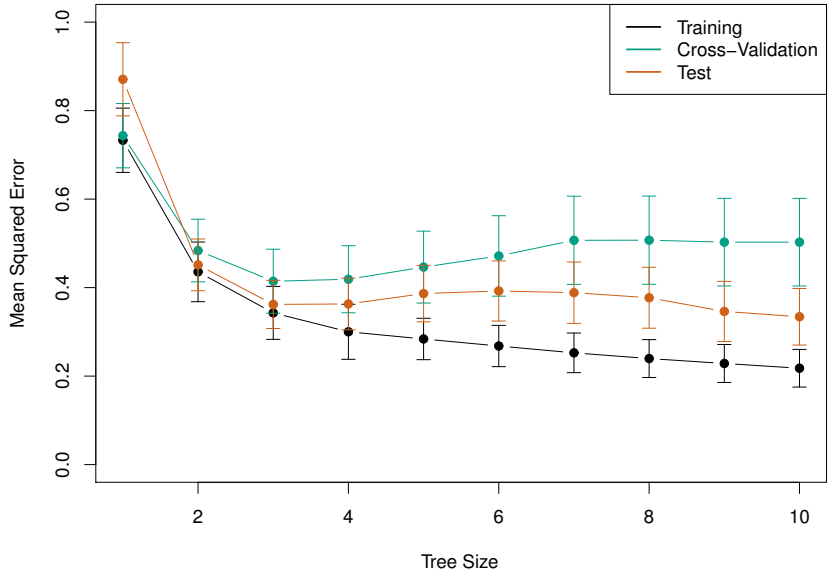
Note: We are doing all fitting, **including the construction of the trees**, using only the training data.

Example. Predicting baseball salaries





Example. Predicting baseball salaries



Classification trees

- They work much like regression trees.
- We predict the response by **majority vote**, i.e. pick the most common class in every region.
- Instead of trying to minimize the RSS:

$$\sum_{m=1}^{|T|} \sum_{x_i \in R_m} (y_i - \bar{y}_{R_m})^2$$

we minimize a classification loss function.

Classification losses

- The 0-1 loss or misclassification rate:

$$\sum_{m=1}^{|T|} \sum_{x_i \in R_m} \mathbf{1}(y_i \neq \hat{y}_{R_m})$$

- The Gini index:

$$\sum_{m=1}^{|T|} q_m \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}),$$

  where $\hat{p}_{m,k}$ is the proportion of class k within R_m, and q_m is the proportion of samples in R_m.

- The cross-entropy:

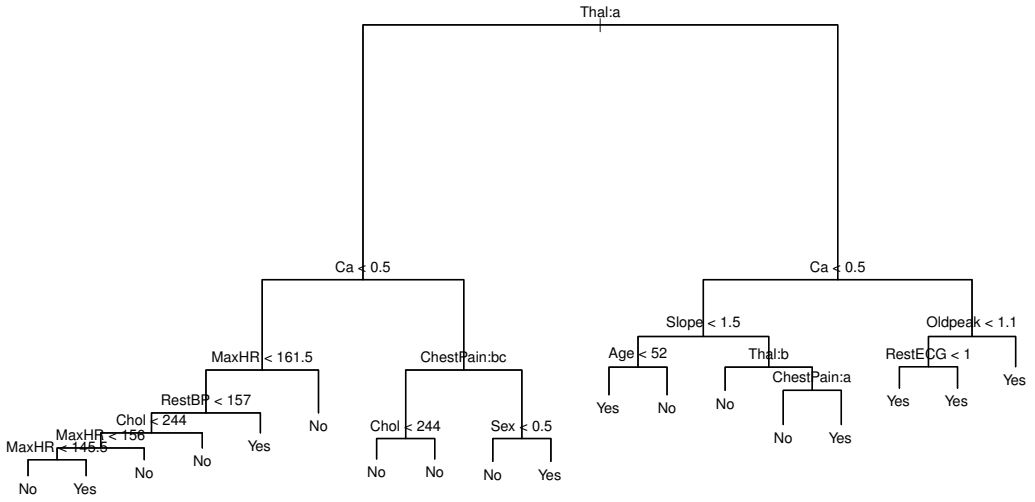
$$-\sum_{m=1}^{|T|} q_m \sum_{k=1}^{K} \hat{p}_{mk} \log(\hat{p}_{mk}).$$

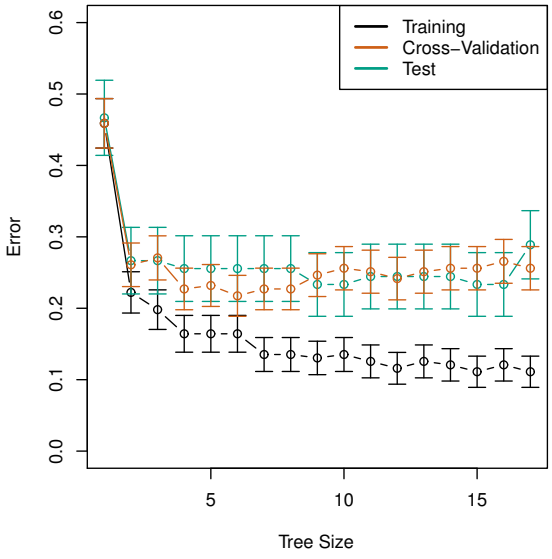
Classification losses

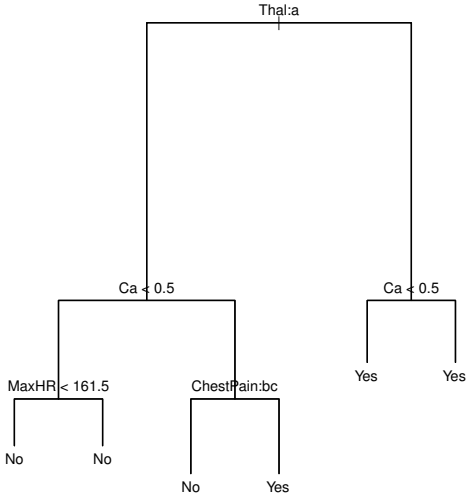
- The Gini index and cross-entropy are better measures of the purity of a region, i.e. they are low when the region is mostly one category.
- **Motivation for the Gini index:**

  If instead of predicting the most likely class, we predict a random sample from the distribution $(\hat{p}_{1,m}, \hat{p}_{2,m}, \ldots, \hat{p}_{K,m})$, the Gini index is the expected misclassification rate.
- It is typical to use the Gini index or cross-entropy for growing the tree, while using the misclassification rate when pruning the tree.

Example. Heart dataset.







Some advantages of decision trees

- Very easy to interpret!
- Closer to human decision-making.
- Easy to visualize graphically.
- They easily handle qualitative predictors and missing data.

R Examples

http://dattahub.github.io/data-analytics/tree_demo_2023.html

Classification and Regression trees, in a nut shell

- Grow the tree by recursively splitting the samples in the leaf R_i according to $X_j > s$, such that (R_i, X_j, s) maximize the drop in RSS.

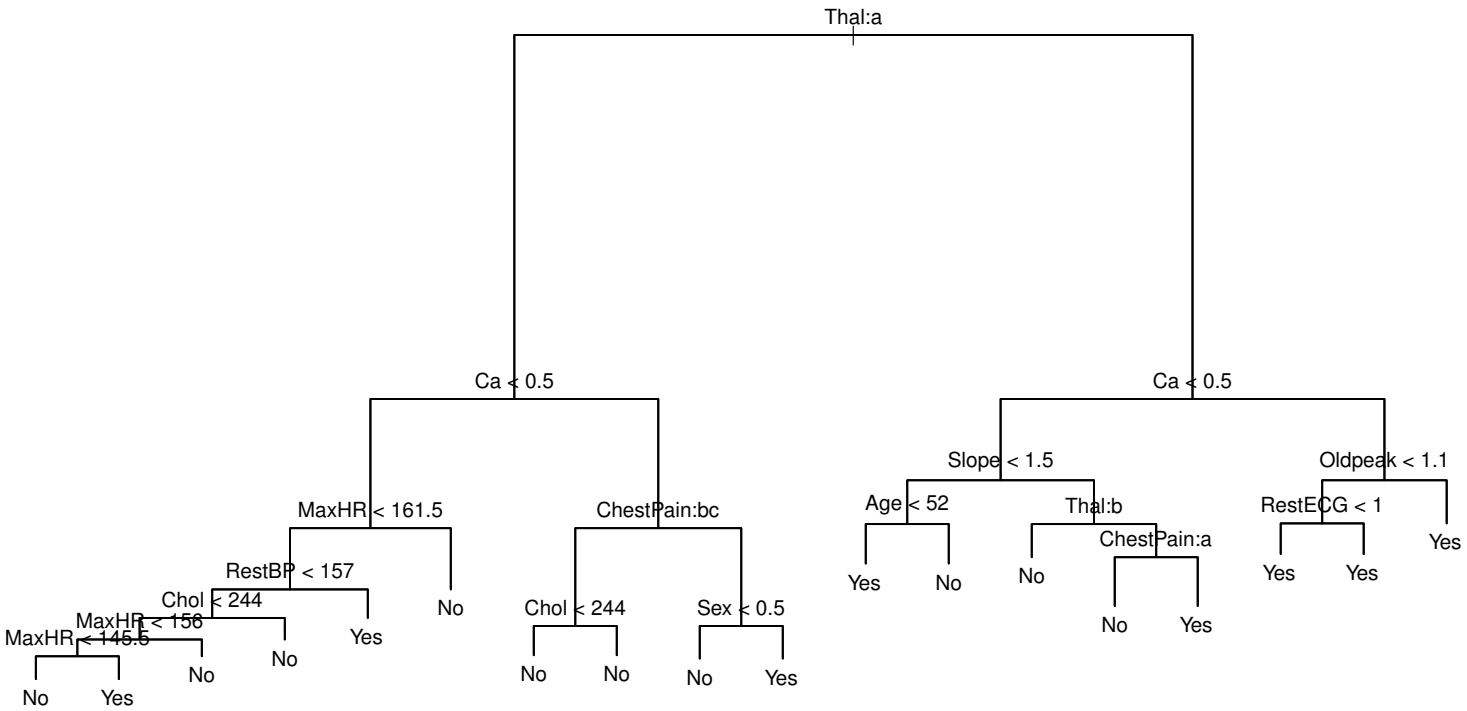
  $\rightarrow$ Greedy algorithm.

- Create a sequence of subtrees $T_0, T_1, \ldots, T_m$ using a **pruning** algorithm.
- Select the best tree T_i (or the best α) by cross validation.

  $\rightarrow$ Why might it be better to choose α instead of the tree T_i by cross-validation?

Example. Heart dataset.

How do we deal with categorical predictors?



Categorical predictors

- If there are only 2 categories, then the split is obvious. We don't have to choose the splitting point s, as for a numerical variable.
- If there are more than 2 categories:
  - Order the categories according to the average of the response:

$$\texttt{ChestPain}:\texttt{a} > \texttt{ChestPain}:\texttt{c} > \texttt{ChestPain}:\texttt{b}$$

  - Treat as a numerical variable with this ordering, and choose a splitting point s.
- One can show that this is the optimal way of partitioning.

Missing data

- Suppose we can assign every sample to a leaf R_i despite the missing data.
- When choosing a new split with variable X_j (growing the tree):
  - Only consider the samples which have the variable X_j.
  - In addition to choosing the best split, choose a second best split using a different variable, and a third best, ...
- To propagate a sample down the tree, if it is missing a variable to make a decision, try the second best decision, or the third best, ...

R Examples

We have looked at the R codes for fitting a decision tree and pruning it for optimality for classification problem.
We will see how it compares to classical methods e.g. logistic for the same data and also look at regression, and (later) the extensions for bagging and random forest.

http://dattahub.github.io/data-analytics/tree_demo_2023.html

Bagging: Bootstrap Aggregating

Let's quickly (re)-visit Bootstrap.

Setup

Suppose we have a model and a fitting method for a particular set of training data.
Denote the training set by

$$Z = (z_1, z_2, \ldots, z_n) \quad \text{where} \quad z_i = (x_i, y_i)$$

in our regression context.
Suppose that you were interested in some statistics $S(Z)$ about an *aspect* of the fit $\hat{f} \mid Z$.
- S may be the variance of some parameter used by f, + i.e., $\hat{\beta}$ in our regression context; - or an aspect of the predictive equations $\hat{f}(x)$, etc.

The method

The idea is to randomly **draw data sets with replacement** from the elements of the training data (Z),
- each of which gives a different realization of S.

$$Z^* \sim Z \quad \Rightarrow \quad S(Z^*) \sim S(Z)$$

- Each sample is typically the same size as the original set (n). - This is repeated B times ($B = 100$ say), producing B **bootstrap datasets**.
The B samples of S thus obtained define a **nonparametric distribution**.
- Summarize $S(Z)$ via the collection $\{S(Z^{*1}), \ldots, S(Z^{*B})\}$, - usually via historgrams or empirical quantiles (estimating CIs).

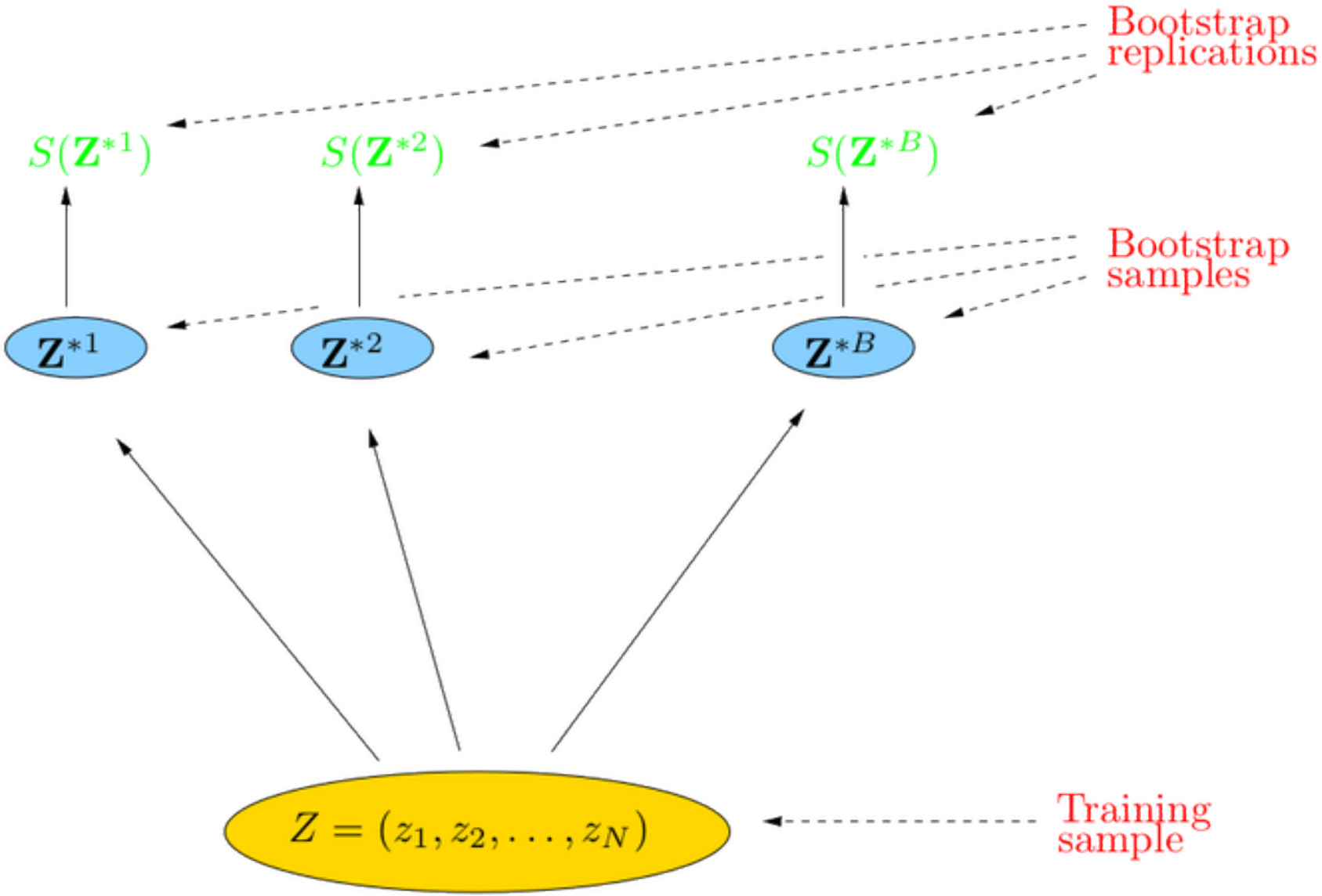


Figure: A bootstrap cartoon

Bootstrap to Bagging

- Bootstrap is a powerful idea.
- It is used in many situations in which it is hard or even impossible to directly compute the sampling distribution of a quantity of interest.
- We see here that the bootstrap can be used in a completely different context, in order to improve statistical learning methods such as decision trees.

Decision Trees

- The decision trees discussed earlier suffer from high variance.
- This means that if we split the training data into two parts at random, and fit a decision tree to both halves, the results that we get could be quite different.
- In contrast, linear regression has low variance if $n \gg p$.
- Bootstrap aggregation, or bagging, is a **general-purpose procedure** for reducing the variance of a statistical learning method.
- Frequently used in decision trees.

Reducing Variance

- Given a set of n independent observations $Z_1, \ldots, Z_n$, each with variance σ^2, the variance of the mean $\bar{Z}$ is σ^2/n.
- In other words, averaging a set of observations reduces variance.
- Hence a natural way to reduce variance would be to
  1. Take many training sets from the population,
  2. build a separate prediction model using each training set, $\hat{f}^1(x), \ldots, \hat{f}^B(x)$ and
  3. average the resulting predictions.

$$\hat{f}_{\text{avg}}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^b(x)$$

Reducing Variance

- Given a set of n independent observations $Z_1, \ldots, Z_n$, each with variance σ^2, the variance of the mean $\bar{Z}$ is σ^2/n.
- In other words, averaging a set of observations reduces variance.
- Hence a natural way to reduce variance would be to
  1. Take many training sets from the population,
  2. build a separate prediction model using each training set, $\hat{f}^1(x), \ldots, \hat{f}^B(x)$ and
  3. average the resulting predictions.

$$\hat{f}_{\text{avg}}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}^b(x)$$

- Of course, this is not practical. We don't have multiple training samples.

Bagging

- Instead, we can bootstrap, by taking repeated samples from the (single) training data set.
- In this approach:
  1. Generate B different bootstrapped training data sets.
  2. train our method on the bth bootstrapped training set in order to get $\hat{f}^{*b}(x)$, and
  3. average all the predictions.

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{*b}(x)$$

- This is called bagging, and you can bag anything, e.g. you can bag Lasso.

Bagging

- Bagging = Bootstrap Aggregating
- In the Bootstrap, we replicate our dataset by sampling with replacement:
  - Original dataset: $\texttt{x} = \texttt{c}(\texttt{x1}, \texttt{x2}, \ldots, \texttt{x100})$
  - Bootstrap samples:
    $\texttt{boot1} = \texttt{sample}(\texttt{x}, 100, \texttt{replace} = \texttt{True})$, ...,
    $\texttt{bootB} = \texttt{sample}(\texttt{x}, 100, \texttt{replace} = \texttt{True})$.
- We used these samples to get the Standard Error of a parameter estimate:

$$\mathsf{SE}(\hat{\beta}_1) \approx \sqrt{\frac{1}{B-1}\sum_{b=1}^{B}(\hat{\beta}_1^{(b)} - \frac{1}{B}\sum_{k=1}^{B}\hat{\beta}_1^{(k)})^2}$$

Bagging

- In **Bagging** we average the predictions of a model fit to many Bootstrap samples.

  Example. Bagging the Lasso

  - Let $\hat{y}^{L,b}$ be the prediction of the Lasso applied to the bth bootstrap sample.
  - Bagging prediction:

$$\hat{y}^{\text{boot}} = \frac{1}{B} \sum_{b=1}^{B} \hat{y}^{L,b}.$$

When does Bagging make sense?

When a regression method or a classifier has a tendency to overfit, Bagging reduces the variance of the prediction.

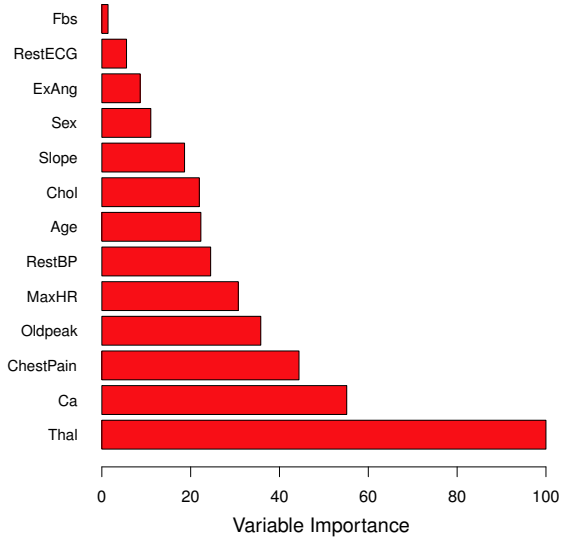
- When n is large, the empirical distribution is similar to the true distribution of the samples.
- Bootstrap samples are like independent realizations of the data.
- Bagging amounts to averaging the fits from many independent datasets, which would reduce the variance by a factor $1/B$.

Bagging decision trees

- **Disadvantage:** Every time we fit a decision tree to a Bootstrap sample, we get a different tree T^b.

  $\rightarrow$ Loss of interpretability

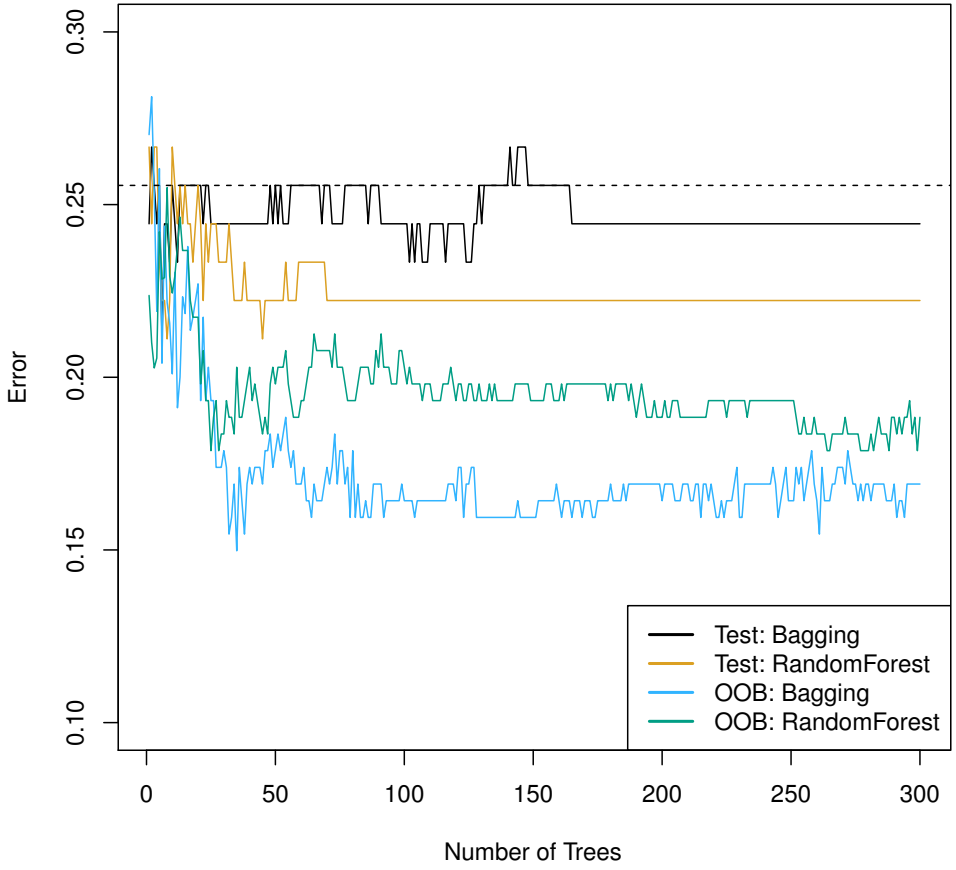
- For each predictor, add up the total amount by which the RSS (or Gini index) decreases every time we use the predictor in T^b.
- Average this total over each Boostrap estimate $T^1, \ldots, T^B$.



Out-of-bag (OOB) error

- To estimate the test error of a bagging estimate, we could use cross-validation.
- Each time we draw a Bootstrap sample, we only use 63% of the observations.
- **Idea:** use the rest of the observations as a test set.
- **OOB error:**
  - For each sample x_i, find the prediction $\hat{y}_i^b$ for all bootstrap samples b which do not contain x_i. There should be around $0.37B$ of them. Average these predictions to obtain $\hat{y}_i^{\text{oob}}$.
  - Compute the error $(y_i - \hat{y}_i^{\text{oob}})^2$.
  - Average the errors over all observations $i = 1, \ldots, n$.

Out-of-bag (OOB) error



The test error decreases as we increase B
(dashed line is the error for a plain decision tree).

Random Forest Intuition

- When building a random forest, at each split in the tree, the algorithm is not even allowed to consider a majority of the available predictors.

Random Forest Intuition

- When building a random forest, at each split in the tree, the algorithm is not even allowed to consider a majority of the available predictors.
- This may sound crazy, but it has a clever rationale.

Random Forest Intuition

- When building a random forest, at each split in the tree, the algorithm is not even allowed to consider a majority of the available predictors.
- This may sound crazy, but it has a clever rationale.
- Suppose that there is one very strong predictor in the data set, along with a number of other moderately strong predictors.

Random Forest Intuition

- When building a random forest, at each split in the tree, the algorithm is not even allowed to consider a majority of the available predictors.
- This may sound crazy, but it has a clever rationale.
- Suppose that there is one very strong predictor in the data set, along with a number of other moderately strong predictors.
- Thus in the collection of bagged trees, most or all of the trees will use this strong predictor in the top split.

Random Forest Intuition

- When building a random forest, at each split in the tree, the algorithm is not even allowed to consider a majority of the available predictors.
- This may sound crazy, but it has a clever rationale.
- Suppose that there is one very strong predictor in the data set, along with a number of other moderately strong predictors.
- Thus in the collection of bagged trees, most or all of the trees will use this strong predictor in the top split.
- Consequently, all of the bagged trees will look quite similar to each other. Hence the predictions from the bagged trees will be highly correlated.

Random Forest Intuition I

- Key idea: averaging many highly correlated quantities does not lead to as large of a reduction in variance as averaging many uncorrelated quantities.
- In particular, this means that bagging will not lead to a substantial reduction in variance over a single tree in this setting.
- Random forests overcome this problem by forcing each split to consider only a subset of the predictors.

Random Forest Intuition II

- Therefore, on average $(p - m)/p$ of the splits will not even consider the strong predictor, and so other predictors will have more of a chance.
- We can think of this process as decorrelating the trees, thereby making the average of the resulting trees less variable and hence more reliable.
- The main difference between bagging and random forests is the choice of predictor subset size m.
- Recall: a random forest is built using $m = p$, then this amounts simply to bagging.
- Using a small value of m in building a random forest will typically be helpful when we have a large number of correlated predictors.

Random Forests

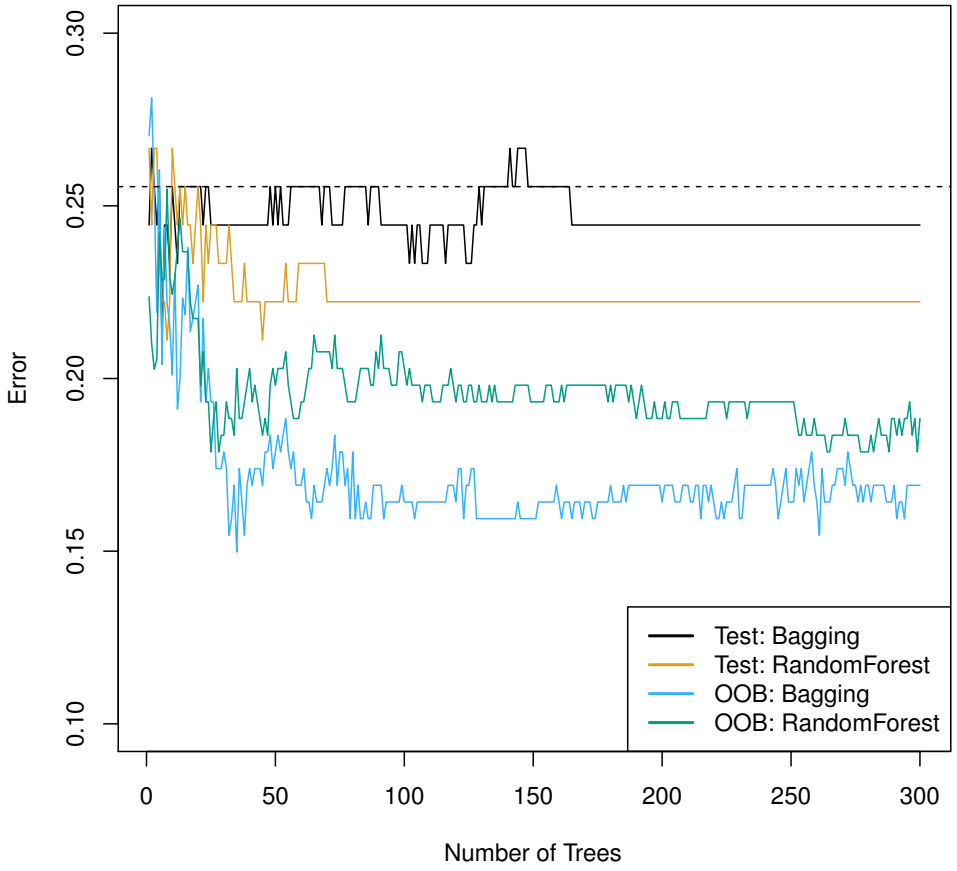
Bagging has a problem:

$\rightarrow$ The trees produced by different Bootstrap samples can be very similar.

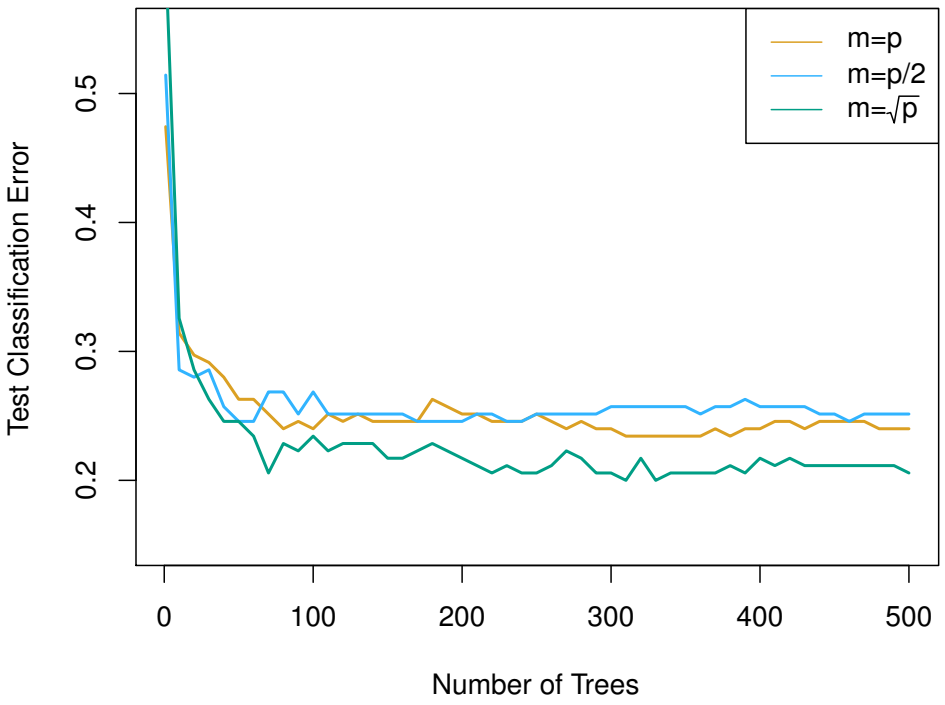
Random Forests:

- We fit a decision tree to different Bootstrap samples.
- When growing the tree, we select a random sample of $m < p$ predictors to consider in each step.
- This will lead to very different (or "uncorrelated") trees from each sample.
- Finally, average the prediction of each tree.

Random Forests vs. Bagging



Random Forests, choosing m



The optimal m is usually around $\sqrt{p}$,
but this can be used as a tuning parameter.

R Examples: Bagging and Random Forest

http://dattahub.github.io/data-analytics/tree_demo_2023.html